CREATING A PASSWORD GRANT CLIENT

In order to use the password grant flow, you need a password grant client in your database. One will have been added when you ran `php artisan passport:install`, but if you ever need to generate a new password grant client for any reason, you can:

```
php artisan passport:client --password

 What should we name the password grant client?
   [My Application Password Grant Client]:
 > SpaceBook_internal

Password grant client created successfully.
```

With the password grant type, there is just one step to getting a token: sending the user's credentials to the `/oauth/token` route, like in Example 13-19.

Example 13-19. Making a request with the password grant type

```
// Assuming SpaceBook is not an external app, but actually
// a trusted internal app... this is SpaceBook's routes/web.php
Route::get('speakr/password-grant-auth', function () {
    $http = new GuzzleHttp\Client;

    $response = $http->post('http://speakr.dev/oauth/token', [
        'form_params' => [
            'grant_type' => 'password',
            'client_id' => config('speakr.id'),
            'client_secret' => config('speakr.secret'),
            'username' => 'matt@mattstauffer.co',
            'password' => 'my-speakr-password',
        ],
    ]);

    $thisUsersTokens = json_decode((string) $response->getBody(), true);
    // do stuff with the tokens
});
```

This route will return an `access_token` and a `refresh_token`. You can now save those tokens to use to authenticate with the API (access token) and to request more tokens later (refresh token).

Note that the ID and secret we'd use for the password grant type would be those in the `clients` database table of our Passport app in the row with the name `SpaceBook_internal`.

Authorization code grant

The most common OAuth 2.0 auth workflow is also the most complex one Passport supports. Let's imagine we're developing an application that's like Twitter but for sound clips; we'll call it Speakr. And we'll imagine another website, a social network for science fiction fans, called SpaceBook. SpaceBook's developer wants to let people embed their Speakr data into their SpaceBook newsfeeds. We're going to install Passport in our app so that other apps — SpaceBook, for example — can allow their mutual users to authenticate with their Speakr information.

In the authorization code grant type, each consuming website — SpaceBook, in this example — needs to create a “client” in our Passport-enabled app. In most scenarios, the other sites’ admins will have user accounts at Speakr, and we’ll build tools for them to create clients there. But for starters, we can just manually create a client for the SpaceBook admins:

```
php artisan passport:client
Which user ID should the client be assigned to?:
 > 1 ❶

 What should we name the client?:
 > SpaceBook

 Where should we redirect the request after authorization?
   [http://passport.dev/auth/callback]:
 > http://spacebook.dev/auth/callback

 New client created successfully.
 Client ID: 3
 Client secret: RiQstsWDqd9SqQY3lQhiZF50ulKdw4iPhPAdkeO3
```

❶ Every client needs to be assigned to a user in your app. Imagine Jill, user #1, is writing SpaceBook; she’ll be the “owner” of this client we’re creating.

Now we have the ID and secret for the SpaceBook client. At this point, SpaceBook can use this ID and secret to build tooling allowing an individual SpaceBook user (who is also a Speakr user) to get an auth token from Speakr for use when SpaceBook wants to make API calls to Speakr on that user’s behalf. Example 13-20 illustrates this. (This and the following examples assume SpaceBook is a Laravel app, too; they also assume we’ve created a file at *config/speakr.php* that returns the ID and secret we just created.)

Example 13-20. A consumer app redirecting a user to our OAuth server

```php
// In SpaceBook's routes/web.php:
Route::get('speakr/redirect', function () {
    $query = http_build_query([
        'client_id' => config('speakr.id'),
        'redirect_uri' => url('speakr/callback'),
        'response_type' => 'code',
    ]);

    // Builds a string like:
    // client_id={$client_id}&redirect_uri={$redirect_uri}&response_type=code

    return redirect('http://speakr.dev/oauth/authorize?' . $query);
});
```

When users hit that route in SpaceBook, they’ll now be redirected to the `/oauth/authorize` Passport route on our Speakr app. At this point they’ll see a confirmation page; you can use the default Passport confirmation page by running this command:

```
php artisan vendor:publish --tag=passport-views
```

This will publish the view to *resources/views/vendor/passport/authorize.blade.php*, and your users will see the page shown in Figure 13-1.

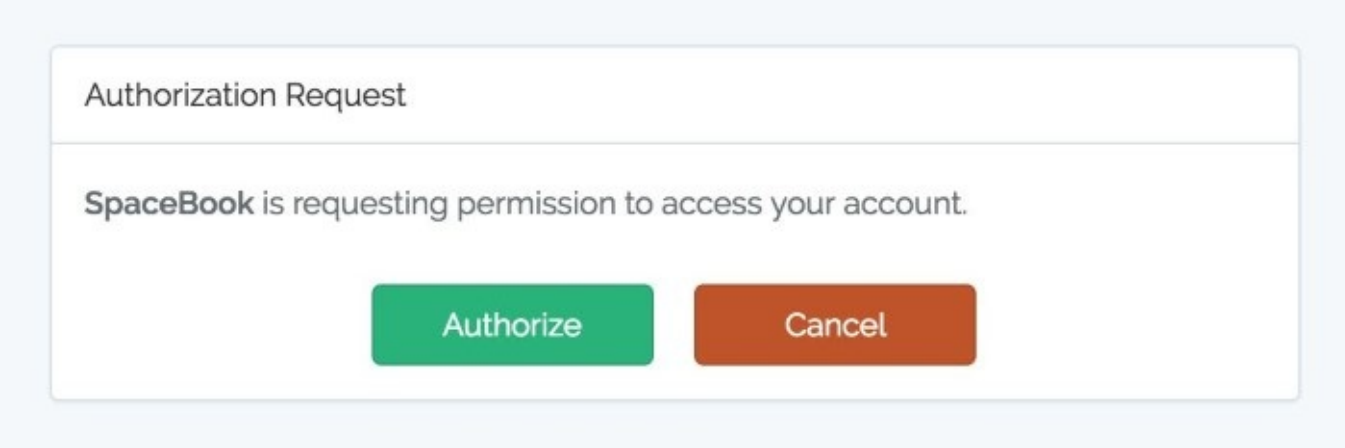

Figure 13-1. OAuth authorization code approval page

Once a user chooses to accept or reject the authorization, Passport will redirect that user back to the provided `redirect_uri`. In Example 13-20 we set a `redirect_uri` of `url('speakr/callback')`, so the user will be redirected back to *http://spacebook.dev/speakr/callback*.

An approval request will contain a code that our consumer app's callback route can now use to get a token back from our Passport-enabled app, Speakr. A rejection request will contain an error. SpaceBook's callback route might look something like Example 13-21.

Example 13-21. The authorization callback route in our sample consuming app

```
// In SpaceBook's routes/web.php:
Route::get('speakr/callback', function (Request $request) {
    if ($request->has('error')) {
        // handle error condition
    }

    $http = new GuzzleHttp\Client;

    $response = $http->post('http://speakr.dev/oauth/token', [
        'form_params' => [
            'grant_type' => 'authorization_code',
            'client_id' => config('speakr.id'),
            'client_secret' => config('speakr.secret'),
            'redirect_uri' => url('speakr/callback'),
            'code' => $request->code,
        ],
    ]);

    $thisUsersTokens = json_decode((string) $response->getBody(), true);
    // do stuff with the tokens
});
```

What we're doing here is building a Guzzle HTTP request to the `/oauth/token` Passport route on Speakr. We send a `POST` request containing the authorization code we received when the user approved access, and Speakr will return a JSON response containing a few keys:

- `access_token` is the token SpaceBook will want to save for this user. This token is what the user will use to authenticate in future requests to Speakr.
- `refresh_token` is a token SpaceBook will need *if* you decide to set your tokens to expire.

By default, Passport's access tokens never need to be refreshed, so you don't need to concern yourself with this and can just ignore it.

- `expires_in` is the number of seconds until an `access_token` expires (needs to be refreshed).

USING REFRESH TOKENS

If you'd like to force users to reauthenticate more often, you need to set a shorter refresh time on the tokens, and then you can use the `refresh_token` to request a new `access_token`.

To set a shorter refresh time:

```
// AuthServiceProvider's boot() method
public function boot()
{
    $this->registerPolicies();

    Passport::routes();

    // How long a token lasts before needing refreshing
    Passport::tokensExpireIn(
        Carbon::now()->addDays(15)
    );

    // How long a refresh token will last before re-auth
    Passport::refreshTokensExpireIn(
        Carbon::now()->addDays(30)
    );
}
```

To request a new token using a refresh token, you need to have first saved the `refresh_token` from the initial auth response in Example 13-21. Once it's time to refresh, you'll make a call similar to that example, but modified slightly:

```
// In SpaceBook's routes/web.php:
Route::get(
    'speakr/request-refresh',
    function (Request $request) {
        $http = new GuzzleHttp\Client;

        $params = [
            'grant_type' => 'refresh_token',
            'client_id' => config('speakr.id'),
            'client_secret' => config('speakr.secret'),
            'redirect_uri' => url('speakr/callback'),
            'refresh_token' => $theTokenYouSavedEarlier,
        ];

        $response = $http->post(
            'http://speakr.dev/oauth/token',
            ['form_params' => $params,]
        );

        $thisUsersTokens = json_decode(
            (string) $response->getBody(),
            true
        );

        // do stuff with the tokens
    }
);
```

In the response, you'll receive a fresh set of tokens to save to your `user`.

You now have all the tools you need to perform basic authorization code flows. We'll cover how to build an admin panel for your clients and tokens later, but first, let's take a quick look at the other grant types.

Personal access tokens

The authorization code grant is great for your users' apps and the password code grant is great for your own apps, but what if your users want to create tokens for themselves to test out your API or to use when they're developing their apps? That's what personal tokens are for.

CREATING A PERSONAL ACCESS CLIENT

In order to create personal tokens, you need a personal access client in your database. Running `php artisan passport:install` will have added one already, but if you ever need to generate a new personal access client for any reason, you can run `php artisan passport:client --personal`:

```
php artisan passport:client --personal

 What should we name the password grant client?
   [My Application Personal Access Client]:
 > My Application Personal Access Client

Personal access client created successfully.
```

Personal access tokens are not quite a "grant" type; there's no OAuth-prescribed flow here. Rather, they're a convenience method that Passport adds to make it easy to have a single client registered in your system that exists solely for the easy creation of convenience tokens for your users who are developers.

For example, maybe you have a user who's developing a competitor to SpaceBook named RaceBook (it's for marathon runners), and he wants to toy around with the Speakr API a bit to figure out how it works *before* starting to code. Does this developer have the facility to create tokens using the authorization code flow? Not yet — he hasn't even written any code yet! That's what personal access tokens are for.

You can create personal access tokens through the JSON API, which I'll cover shortly, but you can also create one for your user directly in code:

```php
// Creating a token without scopes
$token = $user->createToken('Token Name')->accessToken;

// Creating a token with scopes...
$token = $user->createToken('My Token', ['place-orders'])->accessToken;
```

Your users can use these tokens just as if they were tokens created with the authorization code grant flow.

Tokens from Laravel session authentication (synchronizer tokens)

There's one final way for your users to get tokens to access your API, and it's another convenience method that Passport adds but which normal OAuth servers don't provide. This method is for when your users are already authenticated because they've logged in to your Laravel app like normal, and you want the JavaScript on your Laravel app to be able to access the API. It'd be a pain to have to reauthenticate the users with the authorization code or password grant flow, so Laravel provides a helper for that.

If you add the `Laravel\Passport\Http\Middleware\CreateFreshApiToken` middleware to your `web` middleware group (in *app/Http/Kernel.php*), every response Laravel sends to your